

گزارش سوال ۲ عملی - تمرین ۵ - سیستم عامل

آ. ابتدا کانتینر مورد نظر را اجرا کرده و در shell ایجاد شده به ریشه رفته و با مشاهده ساختار آن میبینیم که با ریشه میزبان متفاوت است.

ب. پاسخ: بدیهتا آدرس ریشه کانتینر و میزبان متفاوت اند، چرا که میزبان می خواهد دسترسی و منابع محدود و قابل کنترل به کانتینر بدهد و اگر مسیر ریشه خود را در اختیار کانتینر قرار دهد، کانتینر به تمام سیستم دسترسی خواهد داشت و امکان و توانایی ایجاد تغییر و اختلال را در میزبان خواهد داشت. به همین دلیل میزبان یک آدرس به هر کانتینر اختصاص داده و ریشه آن کانتینر را درون آن پوشه قرار می دهد.

```
bagheri-timarchi@debian:~$ docker run -d --name busybox-sleep busybox sleep 3600
Unable to find image 'busybox:latest' locally
latest: Pulling from library/busybox
e59838ecfec5: Pull complete
Digest: sha256:e3652a00a2fabd16ce889f0aa32c38eec347b997e73bd09e69c962ec7f8732ee
Status: Downloaded newer image for busybox:latest
09b043891bc22d2411d1b541777621715435115ff039fec5458b093e73db760b
bagheri-timarchi@debian:~$ docker exec -it busybox-sleep sh
/ # cd /
/ # pwd
/
/ # ls -la
total 48
drwxr-xr-x  1 root    root      4096 Dec  4 15:24 .
drwxr-xr-x  1 root    root      4096 Dec  4 15:24 ..
-rwxr-xr-x  1 root    root         0 Dec  4 15:24 .dockerenv
drwxr-xr-x  2 root    root     12288 Sep 26  2024 bin
drwxr-xr-x  5 root    root       340 Dec  4 15:24 dev
drwxr-xr-x  1 root    root      4096 Dec  4 15:24 etc
drwxr-xr-x  2 nobody nobody    4096 Sep 26  2024 home
drwxr-xr-x  2 root    root      4096 Sep 26  2024 lib
lrwxrwxrwx  1 root    root         3 Sep 26  2024 lib64 -> lib
dr-xr-xr-x 275 root    root         0 Dec  4 15:24 proc
drwx----- 1 root    root      4096 Dec  4 15:25 root
dr-xr-xr-x 11 root    root         0 Dec  4 15:24 sys
drwxrwxrwt  2 root    root      4096 Sep 26  2024 tmp
drwxr-xr-x  4 root    root      4096 Sep 26  2024 usr
drwxr-xr-x  4 root    root      4096 Sep 26  2024 var
/ #
```

ج. سعی کردیم دستور گفته شده را با کاربر ریشه اجرا کنیم، اما docker daemon کاربر ریشه نمیتوانست آن کانتینر را

ببیند و خروجی زیر را دریافت کردیم:

```
Error: No such object: busybox-sleep
```

همچنین کاربر معمولی نیز با خروجی زیر مواجه می‌شد:

```
null
```

```
root@debian:~# docker inspect busybox-sleep | jq '.[].GraphDriver.Data.MergedDir'
Error: No such object: busybox-sleep
root@debian:~# exit
logout
bagheri-timarchi@debian:~$ docker inspect busybox-sleep | jq '.[].GraphDriver.Data.MergedDir'
null
bagheri-timarchi@debian:~$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS          NAMES
09b043891bc2   busybox       "sleep 3600"            10 minutes ago Up 10 minutes                busybox-sleep
86a1359fda9d   severalnines/sysbench "bash"               4 weeks ago   Exited (0) 4 weeks ago      zealous_mcnulty
65470f67d66e   severalnines/sysbench:latest "bash"               4 weeks ago   Exited (0) 4 weeks ago      sysbench2
42803629e05c   severalnines/sysbench:latest "bash"               5 weeks ago   Exited (0) 5 weeks ago      beautiful_banzai
c1537d54bdc0   severalnines/sysbench:latest "bash"               5 weeks ago   Exited (0) 5 weeks ago      HW2-testbenches
bagheri-timarchi@debian:~$ docker context ls
NAME      DESCRIPTION          DOCKER ENDPOINT          ERROR
default   Current DOCKER_HOST based configuration   unix:///var/run/docker.sock
desktop-linux * Docker Desktop        unix:///home/bagheri-timarchi/.docker/desktop/docker.sock
bagheri-timarchi@debian:~$ docker ps -a | grep busybox
09b043891bc2   busybox       "sleep 3600"            11 minutes ago Up 11 minutes                busybox-sleep
bagheri-timarchi@debian:~$ docker inspect busybox-sleep | jq '.[].GraphDriver.Data.MergedDir'
null
```

چراکه در حال استفاده از docker-desktop بودیم که به طور مستقیم از درایور overlay2 استفاده نمی‌کند و داکر را

درون یک vm پنهان اجرا می‌کند. به همین دلیل میزبان نمی‌تواند MergedDir واقعی را مشاهده کند.

بنابراین ابتدا docker engine را در کنار docker-desktop نصب کردیم:

```
bagheri-timarchi@debian:~$ # Update packages
sudo apt update
sudo apt install -y \
  ca-certificates \
  curl \
  gnupg \
  lsb-release

# Add Docker's official GPG key
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/debian/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg

# Add Docker repository
echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/debian \
  $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# Update again and install Docker Engine
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
Hit:1 http://deb.debian.org/debian trixie InRelease
Get:2 https://download.docker.com/linux/debian trixie InRelease [32.5 kB]
```

و از اجرای آن مطمئن شدیم:

```
bagheri-timarchi@debian:~$ sudo systemctl start docker
sudo systemctl enable docker
Synchronizing state of docker.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable docker
bagheri-timarchi@debian:~$ sudo systemctl status docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled)
   Active: active (running) since Thu 2025-12-04 19:22:44 +0330
     Invocation: 4ec13cb5d6284c52860de3a51014d5e9
   TriggeredBy: ● docker.socket
     Docs: https://docs.docker.com
   Main PID: 17955 (dockerd)
     Tasks: 18
    Memory: 26.4M (peak: 32.3M)
       CPU: 513ms
    CGroup: /system.slice/docker.service
            └─17955 /usr/bin/dockerd -H fd:// --containerd=/run/

Dec 04 19:22:42 debian dockerd[17955]: time="2025-12-04T19:22:42.
Dec 04 19:22:42 debian dockerd[17955]: time="2025-12-04T19:22:42.
Dec 04 19:22:42 debian dockerd[17955]: time="2025-12-04T19:22:42.
Dec 04 19:22:44 debian dockerd[17955]: time="2025-12-04T19:22:44.
Dec 04 19:22:44 debian dockerd[17955]: time="2025-12-04T19:22:44.
Dec 04 19:22:44 debian dockerd[17955]: time="2025-12-04T19:22:44.
Dec 04 19:22:44 debian dockerd[17955]: time="2025-12-04T19:22:44.
Dec 04 19:22:44 debian dockerd[17955]: time="2025-12-04T19:22:44.
Dec 04 19:22:44 debian dockerd[17955]: time="2025-12-04T19:22:44.
Dec 04 19:22:44 debian dockerd[17955]: time="2025-12-04T19:22:44.
Dec 04 19:22:44 debian systemd[1]: Started docker.service - Docker
bagheri-timarchi@debian:~$
```

سپس از docker context را به default تغییر دادیم:

```
bagheri-timarchi@debian:~$ docker context ls
NAME      DESCRIPTION          DOCKER ENDPOINT          ERROR
default   Current DOCKER_HOST based configuration  unix:///var/run/docker.sock
desktop-linux * Docker Desktop      unix:///home/bagheri-timarchi/.docker/desktop/docker.sock

bagheri-timarchi@debian:~$ docker context use default
Current context is now "default"
bagheri-timarchi@debian:~$ docker context ls
NAME      DESCRIPTION          DOCKER ENDPOINT          ERROR
default *   Current DOCKER_HOST based configuration  unix:///var/run/docker.sock
desktop-linux Docker Desktop      unix:///home/bagheri-timarchi/.docker/desktop/docker.sock

bagheri-timarchi@debian:~$ docker info | grep -i "Docker Root Dir"
permission denied while trying to connect to the docker API at unix:///var/run/docker.sock
bagheri-timarchi@debian:~$ sudo docker info | grep -i "Docker Root Dir"
Docker Root Dir: /var/lib/docker
bagheri-timarchi@debian:~$
```

نهایتاً مراحل تمرین را مجدداً انجام دادیم:

```
bagheri-timarchi@debian:~$ sudo docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
bagheri-timarchi@debian:~$ sudo docker run -d --name busybox-sleep busybox sleep 3600
Unable to find image 'busybox:latest' locally
latest: Pulling from library/busybox
e59838ecfec5: Pull complete
Digest: sha256:e3652a00a2fabd16ce889f0aa32c38eec347b997e73bd09e69c962ec7f8732ee
Status: Downloaded newer image for busybox:latest
650e4bf3b321d0371e158601bfb53f003904dc33d446eacf02a80a32080ad47b
bagheri-timarchi@debian:~$ sudo docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
650e4bf3b321  busybox   "sleep 3600"   7 seconds ago   Up 7 seconds   busybox-sleep
bagheri-timarchi@debian:~$ sudo docker inspect busybox-sleep | jq '.[].GraphDriver.Data.MergedDir'
"/var/lib/docker/overlay2/52300f339d45756a2a226ae85393c1cd1c944a71912127c485f916aa5651dd69/merged"
bagheri-timarchi@debian:~$
```

محتوای موجود در این مسیر را با ریشه کانتینر مقایسه میکنیم و مشاهده می کنیم که محتوای این دو یکسان است و آدرس

ریشه کانتینر، آدرسی است که به دست آوردیم و با ریشه میزبان متفاوت است::

```
bagheri-timarchi@debian:~$ sudo docker exec -it busybox-sleep sh
/ # ls -la
total 48
drwxr-xr-x 1 root root 4096 Dec 4 16:07 .
drwxr-xr-x 1 root root 4096 Dec 4 16:07 ..
-rwxr-xr-x 1 root root 0 Dec 4 16:07 .dockerenv
drwxr-xr-x 2 root root 12288 Sep 26 2024 bin
drwxr-xr-x 5 root root 340 Dec 4 16:07 dev
drwxr-xr-x 1 root root 4096 Dec 4 16:07 etc
drwxr-xr-x 2 nobody nobody 4096 Sep 26 2024 home
drwxr-xr-x 2 root root 4096 Sep 26 2024 lib
lrwxrwxrwx 1 root root 3 Sep 26 2024 lib64 -> lib
dr-xr-xr-x 466 root root 0 Dec 4 16:07 proc
drwx----- 1 root root 4096 Dec 4 16:13 root
dr-xr-xr-x 13 root root 0 Dec 4 16:07 sys
drwxrwxrwt 2 root root 4096 Sep 26 2024 tmp
drwxr-xr-x 4 root root 4096 Sep 26 2024 usr
drwxr-xr-x 4 root root 4096 Sep 26 2024 var
/ # exit
bagheri-timarchi@debian:~$ sudo ls -la /var/lib/docker/overlay2/52300f339d45756a2a226ae85393c1cd1c944a71912127c485f916aa5651dd69/merged
total 60
drwxr-xr-x 1 root root 4096 Dec 4 19:37 .
drwx-xr-x 5 root root 4096 Dec 4 19:37 ..
drwxr-xr-x 2 root root 12288 Sep 27 2024 bin
drwxr-xr-x 1 root root 4096 Dec 4 19:37 dev
-rwxr-xr-x 1 root root 0 Dec 4 19:37 .dockerenv
drwxr-xr-x 1 root root 4096 Dec 4 19:37 etc
drwxr-xr-x 2 nobody nogroup 4096 Sep 27 2024 home
drwxr-xr-x 2 root root 4096 Sep 27 2024 lib
lrwxrwxrwx 1 root root 3 Sep 27 2024 lib64 -> lib
drwxr-xr-x 2 root root 4096 Dec 4 19:37 proc
drwx----- 1 root root 4096 Dec 4 19:43 root
drwxr-xr-x 2 root root 4096 Dec 4 19:37 sys
drwxrwxrwt 2 root root 4096 Sep 27 2024 tmp
drwxr-xr-x 4 root root 4096 Sep 27 2024 usr
drwxr-xr-x 4 root root 4096 Sep 27 2024 var
```

مشاهده می‌کنیم در حالت اولیه، از درون کانتینر به دیگر فایل‌های میزبان دسترسی داریم و میتوانیم آن‌ها را مشاهده کنیم:

```
zocker : bash — Konsole
New Tab Split View
bagheri-timarchi@debian:~/Desktop/zocker$ ./zocker run --name cname 'sh'
Running child with pid: 1
$ ls
Makefile README.md src zocker
$ cd ..
$ ls
1404_OS_HW5.pdf      linux-6.1.156_completed.zip  simple
DAST_RANJ            linux-6.1.156.tar.xz        simple.cpp
dns.txt              linux.txt                   tracy
docker.txt           measure_syscalls            tracy-0.12.2.zip
hello_world_kernel   measure_syscalls.cpp        tracy_gui
hello_world_kernel.c OS_Homeworks                 v2rayx.desktop
HW5P-Q2              'part 2'                    'vm testbench result.txt'
'kill kvm.txt'        Q3                           zocker
linux-6.1.156         'report part 2.txt'
$ cd ..
$ ls
Desktop      Music                                soccerengine
Documents    Oracle_VirtualBox_Extension_Pack-7.1.14.vbox-extpack  Templates
Downloads    Pictures                               tracy_build_gui
HW2-testbench Public                                   Videos
libinput-gestures snap                                   'VirtualBox VMs'
$ cd Desktop/zocker
$ exit
sh: 7: Cannot set tty process group (No such process)
[Parent] Stopping...
bagheri-timarchi@debian:~/Desktop/zocker$ rm -rf /tmp/zocker/cname
bagheri-timarchi@debian:~/Desktop/zocker$
```

کرده و سپس با دستور `chdir` به آن می‌رویم:

[illegible]

```
bagheri-timarchi@debian:~/Desktop/zocker$ sudo ./zocker run --name test-container 'sh'
[ERR] Failed to remount /proc: No such file or directory
[ERR] Running container failed due to some internal errors.
[Parent] Stopping...
bagheri-timarchi@debian:~/Desktop/zocker$
```

ایجاد کنیم. برای این کار یک خط `mkdir` را به کد قبلی اضافه می‌کنیم:

و. پس از اجرای دوباره، همانطور که پیش بینی می شد با خطای زیر مواجه شدیم:

6 | Page

پاسخ: علت این خطا این است که ما پس از تغییر مسیر ریشه و قطع دسترسی به فایل های میزبان، قصد اجرای /bin/sh/ را داریم که در ریشه جدید وجود ندارد و حتی اگر آن را ایجاد کنیم، کتابخانه های لازم برای اجرای آن موجود نیست. برای حل خطای بالا، توابع setup_bin_dir و setup_lib_dir فایل setup.c را به شکل زیر تغییر میدهیم:

```
// updated function
static int setup_bin_dir(const char container_dir[256]) {
    char bin_dir[256];
    char cmd[512];

    if (snprintf(bin_dir, sizeof(bin_dir), "%s/bin", container_dir) < 0) {
        return 1;
    }

    mkdir(bin_dir, 0755);

    // Copy /bin/sh into container/bin/sh
    snprintf(cmd, sizeof(cmd), "cp /bin/sh %s/sh", bin_dir);
    if (system(cmd) != 0) {
        fprintf(stderr, "[ERR] Failed to copy /bin/sh into container\n");
        return 1;
    }

    // Copy /bin/ls into container/bin/ls
    snprintf(cmd, sizeof(cmd), "cp /bin/ls %s/ls", bin_dir);
    if (system(cmd) != 0) {
        fprintf(stderr, "[ERR] Failed to copy /bin/ls into container\n");
    }

    // Copy other common commands (e.g., cat, echo, etc.)
    snprintf(cmd, sizeof(cmd), "cp /bin/cat %s/cat", bin_dir);
    system(cmd);

    snprintf(cmd, sizeof(cmd), "cp /bin/pwd %s/pwd", bin_dir);
    system(cmd);

    return 0;
}
```

```

// updated function
static int setup_lib_dir(const char container_dir[256]) {
    char lib_dir[256];
    char lib64_dir[256];
    char cmd[512];
    FILE *fp;
    char line[512];

    snprintf(lib_dir, sizeof(lib_dir), "%s/lib", container_dir);
    mkdir(lib_dir, 0755);

    snprintf(lib64_dir, sizeof(lib64_dir), "%s/lib64", container_dir);
    mkdir(lib64_dir, 0755);

    // EXPLICITLY COPY DYNAMIC LINKER
    snprintf(cmd, sizeof(cmd), "cp -L /lib64/ld-linux-x86-64.so.2 %s/", lib64_dir);
    if (system(cmd) != 0) {
        fprintf(stderr, "[ERR] Failed to copy dynamic linker into container\n");
        return 1;
    }

    // Run ldd on all essential binaries copied in setup_bin_dir
    fp = popen("ldd /bin/sh /bin/ls /bin/cat /bin/pwd", "r");
    if (!fp) {
        fprintf(stderr, "[ERR] Failed to run ldd\n");
        return 1;
    }

    while (fgets(line, sizeof(line), fp)) {
        char *path_start = line;

        while (*path_start == ' ' || *path_start == '\t') {
            path_start++;
        }

        size_t len = strlen(path_start);
        if (len > 1 && path_start[len - 2] != ':') { // -2 accounts for potential newline
            continue; // Skip the line entirely
        }

        char *path = strstr(line, "=>");
        if (!path) {
            // some lines have format: "/lib64/ld-linux-x86-64.so.2 (0x...)"
            if (line[0] == '/') {
                path = strtok(line, " "); // strip address
            } else {
                continue;
            }
        } else {
            // standard format: "library.so => /path/to/library.so (0x...)"
            path = strstr(line, "/");
        }

        if (!path) continue;

        path[strcspn(path, " \n")] = '\0'; // trim newline/spaces

        // Decide if it goes to lib or lib64 (Prioritize lib64 for 64-bit libs)
        const char *dest = strstr(path, "lib64") ? lib64_dir : lib_dir;

        // Use -L to dereference symlinks and copy the actual file
        snprintf(cmd, sizeof(cmd), "cp -L %s %s/", path, dest);

        system(cmd);
    }

    pclose(fp);
    return 0;
}

```


ز. نهایتاً مشاهده می‌کنیم که کانتینر بدون هیچ خطایی اجرا می‌شود و با استفاده از دستورات `ls`، `pwd` و `cd` مشاهده می‌کنیم که ریشه کانتینر جدا شده است:

```
bagheri-timarchi@debian:~/Desktop/zocker$ sudo ./zocker run --name test-container 'sh'
Running child with pid: 1
# pwd
/
# ls
bin lib lib64 proc
# cd bin
# ls
cat ls pwd sh
# pwd
/bin
# cd ../lib
# ls
libc.so.6 libcap.so.2 libpcrc2-8.so.0 libselinux.so.1
# pwd
/lib
# cd ..
# pwd
/
```